MODULE 06

React + TypeScript Fundamentals

Everything from Modules 1-5 now applied directly to React. You already know JSX — this module is purely about adding types to what you already build every day.

~60 min read | Needs Modules 1-5 | The React TS transition

1. Setting Up a React TypeScript Project

The easiest way to start is with Vite (the modern replacement for Create React App):

Terminal — Create React TS project

```
# Create a new React + TypeScript project with Vite
npm create vite@latest my-app -- --template react-ts
cd my-app
npm install
npm run dev

# Or with Create React App (older, still works)
npx create-react-app my-app --template typescript
```

Key file differences from a JS React project:

React JS	React TS
App.jsx	App.tsx
main.jsx	main.tsx
.babelrc / babel.config.js	tsconfig.json
No type checking	Full TS compiler checking
PropTypes (optional runtime)	Interfaces (compile-time)

2. Typing Component Props — The Most Important Skill

In React JS you either use PropTypes or just hope the right values are passed in. In React TS, you define an interface for your props — TypeScript enforces them everywhere the component is used.

Basic Props Interface

Button.tsx — typed component props

```tsx
// Button.tsx

interface ButtonProps {
  label:     string;
  onClick:   () => void;
  disabled?: boolean;          // optional
  variant?:  'primary' | 'secondary' | 'danger';  // union
}

// Two equivalent ways to define a functional component:

// Option 1: Inline destructuring with interface (most common)
function Button({ label, onClick, disabled = false, variant = 'primary' }:
ButtonProps) {
  return (
    <button onClick={onClick} disabled={disabled} className={variant}>
      {label}
    </button>
  );
}

// Option 2: React.FC type (less popular but valid)
const Button: React.FC<ButtonProps> = ({ label, onClick }) => {
  return <button onClick={onClick}>{label}</button>;
};

// Usage — TS checks props at every call site
<Button label='Save' onClick={handleSave} />            // OK
<Button label='Save' />                                 // Error: onClick
missing
<Button label='Save' onClick={handleSave} size='lg' /> // Error: 'size'
not in props
```

3. Typing the children Prop

The children prop is special in React. Here are the correct types for different use cases:

Typing the children prop

```
import { ReactNode, ReactElement, PropsWithChildren } from 'react';

// ReactNode — accepts ANYTHING React can render
// strings, numbers, JSX, arrays, null, undefined, fragments
// This is the correct type for 'children' in 99% of cases
interface CardProps {
  title:    string;
  children: ReactNode;   // <-- use this
}

// Shorthand using PropsWithChildren utility
// PropsWithChildren<T> = T & { children?: ReactNode }
interface CardBaseProps { title: string; }
type CardProps2 = PropsWithChildren<CardBaseProps>;

function Card({ title, children }: CardProps) {
  return (
    <div>
      <h2>{title}</h2>
      <div>{children}</div>
    </div>
  );
}

// ReactElement — only JSX (not strings/numbers)
// Use when you need to call React.cloneElement(children)
interface WrapperProps { children: ReactElement; }
```

> **ReactNode vs ReactElement vs JSX.Element**
>
> ReactNode is the widest — accepts anything React can render. ReactElement is narrower — JSX only (not strings). JSX.Element is the return type of a JSX expression specifically. Use ReactNode for children props. Use JSX.Element or ReactElement when you need to clone or inspect the children.

4. Typing Event Handlers

This trips up almost everyone coming from React JS. Events in React TS need specific types from React's type definitions — not the browser's native DOM event types:

React event types

```
import { ChangeEvent, MouseEvent, FormEvent, KeyboardEvent } from 'react';

// Input change event
function handleChange(e: ChangeEvent<HTMLInputElement>): void {
  console.log(e.target.value);   // string
  console.log(e.target.checked); // boolean (for checkboxes)
}

// Button click event
function handleClick(e: MouseEvent<HTMLButtonElement>): void {
  e.preventDefault();
  console.log(e.currentTarget.id);
}

// Form submit
function handleSubmit(e: FormEvent<HTMLFormElement>): void {
  e.preventDefault();
}

// Keyboard event
function handleKeyDown(e: KeyboardEvent<HTMLInputElement>): void {
  if (e.key === 'Enter') { /* submit */ }
}

// In JSX — inline arrow function (TS infers the type!)
<input onChange={(e) => console.log(e.target.value)} />
// TS infers e as ChangeEvent<HTMLInputElement> automatically
```

Event	Type	Common Element
onChange (input/select)	ChangeEvent<T>	HTMLInputElement
onClick	MouseEvent<T>	HTMLButtonElement
onSubmit (form)	FormEvent<T>	HTMLFormElement
onKeyDown/Up/Press	KeyboardEvent<T>	HTMLInputElement
onFocus/onBlur	FocusEvent<T>	HTMLInputElement
onDragStart/Drop	DragEvent<T>	HTMLDivElement

5. Typing useState

useState is generic — it accepts a type parameter. Most of the time TypeScript infers it automatically from the initial value, but you need to be explicit when the state can be more than one type:

useState with TypeScript

```
import { useState } from 'react';

// TS infers these automatically — no annotation needed
const [count,   setCount]   = useState(0);        // number
const [name,    setName]    = useState('');       // string
const [visible, setVisible] = useState(false);    // boolean

// Explicit type needed when initial value is null or ambiguous
const [user,    setUser]    = useState<User | null>(null);
const [items,   setItems]   = useState<string[]>([]);
const [status,  setStatus]  = useState<'idle' | 'loading' |
'done'>('idle');

// Using an interface as state shape
interface FormData {
  email:    string;
  password: string;
}
const [form, setForm] = useState<FormData>({ email: '', password: '' });

// setForm is typed — TS catches mistakes
setForm({ email: 'a@a.com', password: '123' }); // OK
setForm({ email: 'a@a.com' });                  // Error: password missing
setForm({ email: 123 });                        // Error: number not
string
```

6. Typing useRef

useRef has two main use cases in React — DOM refs and mutable value storage. Each needs a different type:

useRef with TypeScript

```
import { useRef } from 'react';

// Use case 1: DOM ref — pass null as initial value
// The generic type is the HTML element type
```

```
const inputRef  = useRef<HTMLInputElement>(null);
const divRef    = useRef<HTMLDivElement>(null);
const buttonRef = useRef<HTMLButtonElement>(null);

// When accessing .current, check for null (element may not be mounted)
function focusInput() {
  inputRef.current?.focus();  // optional chaining handles null
  // OR
  if (inputRef.current) {
    inputRef.current.focus(); // TS narrows: current is HTMLInputElement
here
  }
}

// Use case 2: Mutable value (like instance variable, no re-render)
const timerRef    = useRef<number | null>(null);
const prevValueRef = useRef<string>('');

// Mutable ref — .current can be set directly
timerRef.current = window.setTimeout(() => {}, 1000);

// In JSX — TypeScript checks the element type matches
<input ref={inputRef} />   // OK — inputRef is HTMLInputElement
<div   ref={inputRef} />   // Error — div vs input type mismatch
```

7. Component Return Types

React components return JSX. TypeScript has specific types for this:

Component return types

```
import { JSX, ReactElement, ReactNode } from 'react';

// JSX.Element — what JSX expressions evaluate to
function MyComponent(): JSX.Element {
  return <div>Hello</div>;
}

// ReactElement — same as JSX.Element, slightly more explicit
function MyComponent2(): ReactElement {
  return <div>Hello</div>;
}

// ReactNode — widest type, also allows null/undefined
```

```
// Use when component can conditionally render nothing
function ConditionalComponent({ show }: { show: boolean }): ReactNode {
  if (!show) return null;  // OK because ReactNode includes null
  return <div>Visible!</div>;
}

// In practice: most devs skip the return type annotation
// TS infers it correctly from the JSX. Only annotate if needed.
function Simple() {
  return <div>TS infers JSX.Element automatically</div>;
}
```

8. Extending Native HTML Props

One of the most useful React TS patterns — letting your component accept all native HTML attributes PLUS your own custom props. Essential for building reusable UI components:

Extending HTML element props

```
import { ButtonHTMLAttributes, InputHTMLAttributes } from 'react';

// Extend HTMLButtonElement props — your Button accepts ALL native button
attrs
interface ButtonProps extends ButtonHTMLAttributes<HTMLButtonElement> {
  variant?: 'primary' | 'secondary' | 'danger';  // your custom props
  loading?: boolean;
  // You do NOT need to redeclare onClick, disabled, type, etc.
  // They come from ButtonHTMLAttributes!
}

function Button({ variant = 'primary', loading, children, ...rest }:
ButtonProps) {
  return (
    <button
      {...rest}           // spread all native props (onClick, disabled,
etc.)
      className={variant}
      disabled={loading || rest.disabled}
    >
      {loading ? 'Loading...' : children}
    </button>
  );
}
```

```
// Now your Button accepts ALL native button attributes:
<Button onClick={fn} disabled={false} type='submit' variant='primary'>
  Save Changes
</Button>
```

9. Putting It All Together — A Real Typed Component

A realistic form component combining everything from this module: props interface, children, events, useState, and useRef all working together:

LoginForm.tsx — complete typed component

```
// LoginForm.tsx — a complete typed React component

import { useState, useRef, ChangeEvent, FormEvent } from 'react';

interface LoginFormProps {
  onSuccess:   (token: string) => void;
  onError?:    (message: string) => void;  // optional callback
  redirectUrl?: string;
}

interface FormState {
  email:    string;
  password: string;
}

function LoginForm({ onSuccess, onError }: LoginFormProps) {
  const [form,    setForm]   = useState<FormState>({ email: '', password:
'' });
  const [loading, setLoading] = useState(false);
  const emailRef = useRef<HTMLInputElement>(null);

  const handleChange = (e: ChangeEvent<HTMLInputElement>): void => {
    setForm(prev => ({ ...prev, [e.target.name]: e.target.value }));
  };

  const handleSubmit = async (e: FormEvent<HTMLFormElement>):
Promise<void> => {
    e.preventDefault();
    setLoading(true);
    try {
      const res = await fetch('/api/login', {
        method: 'POST', body: JSON.stringify(form),
```

```
      });
      const { token } = await res.json();
      onSuccess(token);
    } catch (err) {
      onError?.(String(err));  // optional chaining on optional prop
    } finally {
      setLoading(false);
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input ref={emailRef} name='email'    type='email'
onChange={handleChange} />
      <input              name='password' type='password'
onChange={handleChange} />
      <button type='submit' disabled={loading}>
        {loading ? 'Logging in...' : 'Login'}
      </button>
    </form>
  );
}

export default LoginForm;
```

Module Summary

Everything you learned in this module:

- Use Vite (react-ts template) to scaffold a new React TS project
- Define props with an interface: `interface ButtonProps { label: string }`
- Type children as `ReactNode` — accepts anything React can render
- Event types: `ChangeEvent<HTMLInputElement>`, `MouseEvent<HTMLButtonElement>`, `FormEvent<HTMLFormElement>`
- Inline JSX events — TypeScript infers the event type automatically
- useState: `useState<User | null>(null)` — be explicit when initial value is null/ambiguous
- useRef DOM refs: `useRef<HTMLInputElement>(null)` — always check .current for null
- Component return type: `JSX.Element` or `ReactNode` (or let TS infer it)
- Extend native HTML props: `interface ButtonProps extends ButtonHTMLAttributes<HTMLButtonElement>`

> Next: Module 7

React Hooks & Context in TypeScript

Typing useReducer, useContext, useMemo, useCallback, and custom hooks. Plus how to type React Context correctly so the entire context chain is fully type-safe.